10.2 Hash Tables

In this section, we introduce one of the most efficient data structures for implementing a map, and the one that is used most in practice. This structure is known as a ***hash table***.

Intuitively, a map M supports the abstraction of using keys as "addresses" that help locate an entry. As a mental warm-up, consider a restricted setting in which a map with n entries uses keys that are known to be integers in a range from 0 to $N-1$ for some $N \geq n$. In this case, we can represent the map using a ***lookup table*** of length N, as diagrammed in Figure 10.3.

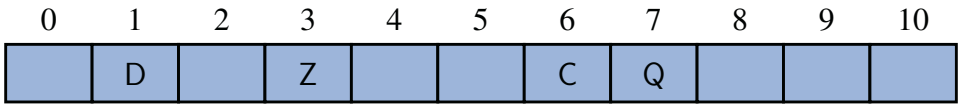

Figure 10.3: A lookup table with length 11 for a map containing entries (1,D), (3,Z), (6,C), and (7,Q).

In this representation, we store the value associated with key k at index k of the table (presuming that we have a distinct way to represent an empty slot). Basic map operations get, put, and remove can be implemented in $O(1)$ worst-case time.

There are two challenges in extending this framework to the more general setting of a map. First, we may not wish to devote an array of length N if it is the case that $N \gg n$. Second, we do not in general require that a map's keys be integers. The novel concept for a hash table is the use of a ***hash function*** to map general keys to corresponding indices in a table. Ideally, keys will be well distributed in the range from 0 to $N-1$ by a hash function, but in practice there may be two or more distinct keys that get mapped to the same index. As a result, we will conceptualize our table as a ***bucket array***, as shown in Figure 10.4, in which each bucket may manage a collection of entries that are sent to a specific index by the hash function. (To save space, an empty bucket may be replaced by a null reference.)

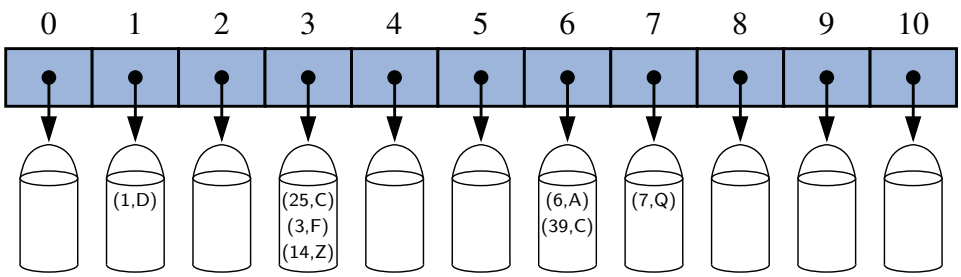

Figure 10.4: A bucket array of capacity 11 with entries (1,D), (25,C), (3,F), (14,Z), (6,A), (39,C), and (7,Q), using a simple hash function.

10.2.1 Hash Functions

The goal of a ***hash function***, h, is to map each key k to an integer in the range $[0, N-1]$, where N is the capacity of the bucket array for a hash table. Equipped with such a hash function, h, the main idea of this approach is to use the hash function value, $h(k)$, as an index into our bucket array, A, instead of the key k (which may not be appropriate for direct use as an index). That is, we store the entry (k, v) in the bucket $A[h(k)]$.

If there are two or more keys with the same hash value, then two different entries will be mapped to the same bucket in A. In this case, we say that a ***collision*** has occurred. To be sure, there are ways of dealing with collisions, which we will discuss later, but the best strategy is to try to avoid them in the first place. We say that a hash function is "good" if it maps the keys in our map so as to sufficiently minimize collisions. For practical reasons, we also would like a hash function to be fast and easy to compute.

It is common to view the evaluation of a hash function, $h(k)$, as consisting of two portions—a ***hash code*** that maps a key k to an integer, and a ***compression function*** that maps the hash code to an integer within a range of indices, $[0, N-1]$, for a bucket array. (See Figure 10.5.)

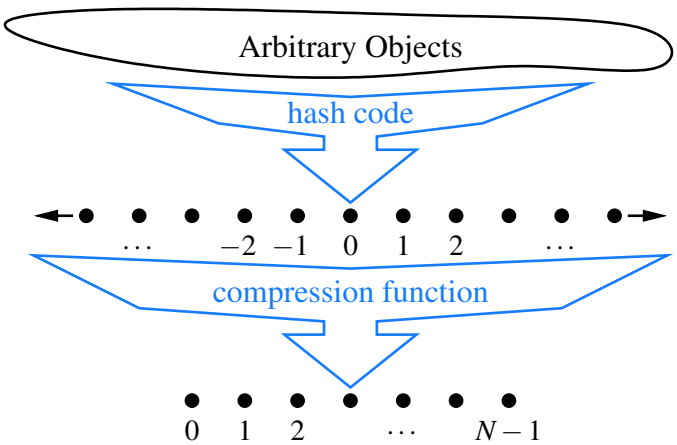

Figure 10.5: Two parts of a hash function: a hash code and a compression function.

The advantage of separating the hash function into two such components is that the hash code portion of that computation is independent of a specific hash table size. This allows the development of a general hash code for each object that can be used for a hash table of any size; only the compression function depends upon the table size. This is particularly convenient, because the underlying bucket array for a hash table may be dynamically resized, depending on the number of entries currently stored in the map. (See Section 10.2.3.)

Hash Codes

The first action that a hash function performs is to take an arbitrary key k in our map and compute an integer that is called the ***hash code*** for k; this integer need not be in the range $[0, N-1]$, and may even be negative. We desire that the set of hash codes assigned to our keys should avoid collisions as much as possible. For if the hash codes of our keys cause collisions, then there is no hope for our compression function to avoid them. In this subsection, we begin by discussing the theory of hash codes. Following that, we discuss practical implementations of hash codes in Java.

Treating the Bit Representation as an Integer

To begin, we note that, for any data type X that is represented using at most as many bits as our integer hash codes, we can simply take as a hash code for X an integer interpretation of its bits. Java relies on 32-bit hash codes, so for base types **byte**, **short**, **int**, and **char**, we can achieve a good hash code simply by casting a value to **int**. Likewise, for a variable x of base type **float**, we can convert x to an integer using a call to Float.floatToIntBits(x), and then use this integer as x's hash code.

For a type whose bit representation is longer than a desired hash code (such as Java's **long** and **double** types), the above scheme is not immediately applicable. One possibility is to use only the high-order 32 bits (or the low-order 32 bits). This hash code, of course, ignores half of the information present in the original key, and if many of the keys in our map only differ in these bits, then they will collide using this simple hash code.

A better approach is to combine in some way the high-order and low-order portions of a 64-bit key to form a 32-bit hash code, which takes all the original bits into consideration. A simple implementation is to add the two components as 32-bit numbers (ignoring overflow), or to take the exclusive-or of the two components. These approaches of combining components can be extended to any object x whose binary representation can be viewed as an n-tuple $(x_0, x_1, \ldots, x_{n-1})$ of 32-bit integers, for example, by forming a hash code for x as $\sum_{i=0}^{n-1} x_i$, or as $x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}$, where the $\oplus$ symbol represents the bitwise exclusive-or operation (which is the ^ operator in Java).

Polynomial Hash Codes

The summation and exclusive-or hash codes, described above, are not good choices for character strings or other variable-length objects that can be viewed as tuples of the form $(x_0, x_1, \ldots, x_{n-1})$, where the order of the x_i's is significant. For example, consider a 16-bit hash code for a character string s that sums the Unicode values of the characters in s. This hash code unfortunately produces lots of unwanted

collisions for common groups of strings. In particular, "temp01" and "temp10" collide using this function, as do "stop", "tops", "pots", and "spot". A better hash code should somehow take into consideration the positions of the x_i's. An alternative hash code, which does exactly this, is to choose a nonzero constant, $a \neq 1$, and use as a hash code the value

$$x_0 a^{n-1} + x_1 a^{n-2} + \cdots + x_{n-2} a + x_{n-1}.$$

Mathematically speaking, this is simply a polynomial in a that takes the components $(x_0, x_1, \ldots, x_{n-1})$ of an object x as its coefficients. This hash code is therefore called a ***polynomial hash code***. By Horner's rule (see Exercise C-4.54), this polynomial can be computed as

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_2 + a(x_1 + a x_0)) \cdots)).$$

Intuitively, a polynomial hash code uses multiplication by different powers as a way to spread out the influence of each component across the resulting hash code.

Of course, on a typical computer, evaluating a polynomial will be done using the finite bit representation for a hash code; hence, the value will periodically overflow the bits used for an integer. Since we are more interested in a good spread of the object x with respect to other keys, we simply ignore such overflows. Still, we should be mindful that such overflows are occurring and choose the constant a so that it has some nonzero, low-order bits, which will serve to preserve some of the information content even as we are in an overflow situation.

We have done some experimental studies that suggest that 33, 37, 39, and 41 are particularly good choices for a when working with character strings that are English words. In fact, in a list of over 50,000 English words formed as the union of the word lists provided in two variants of Unix, we found that taking a to be 33, 37, 39, or 41 produced fewer than 7 collisions in each case!

Cyclic-Shift Hash Codes

A variant of the polynomial hash code replaces multiplication by a with a cyclic shift of a partial sum by a certain number of bits. For example, a 5-bit cyclic shift of the 32-bit value 00111101100101101010100010101000 is achieved by taking the leftmost five bits and placing those on the rightmost side of the representation, resulting in 10110010110101010001010100000111. While this operation has little natural meaning in terms of arithmetic, it accomplishes the goal of varying the bits of the calculation. In Java, a cyclic shift of bits can be accomplished through careful use of the bitwise shift operators.

An implementation of a cyclic-shift hash code computation for a character string in Java appears as follows:

```
static int hashCode(String s) {
  int h=0;
  for (int i=0; i<s.length( ); i++) {
    h = (h << 5) | (h >>> 27);              // 5-bit cyclic shift of the running sum
    h += (int) s.charAt(i);                 // add in next character
  }
  return h;
}
```

As with the traditional polynomial hash code, fine-tuning is required when using a cyclic-shift hash code, as we must wisely choose the amount to shift by for each new character. Our choice of a 5-bit shift is justified by experiments run on a list of just over 230,000 English words, comparing the number of collisions for various shift amounts (see Table 10.1).

	Collisions	
Shift	**Total**	**Max**
0	234735	623
1	165076	43
2	38471	13
3	7174	5
4	1379	3
5	190	3
6	502	2
7	560	2
8	5546	4
9	393	3
10	5194	5
11	11559	5
12	822	2
13	900	4
14	2001	4
15	19251	8
16	211781	37

Table 10.1: Comparison of collision behavior for the cyclic-shift hash code as applied to a list of 230,000 English words. The "Total" column records the total number of words that collide with at least one other, and the "Max" column records the maximum number of words colliding at any one hash code. Note that with a cyclic shift of 0, this hash code reverts to the one that simply sums all the characters.

Hash Codes in Java

The notion of hash codes are an integral part of the Java language. The Object class, which serves as an ancestor of all object types, includes a default hashCode( ) method that returns a 32-bit integer of type **int**, which serves as an object's hash code. The default version of hashCode( ) provided by the Object class is often just an integer representation derived from the object's memory address.

However, we must be careful if relying on the default version of hashCode( ) when authoring a class. For hashing schemes to be reliable, it is imperative that any two objects that are viewed as "equal" to each other have the same hash code. This is important because if an entry is inserted into a map, and a later search is performed on a key that is considered equivalent to that entry's key, the map must recognize this as a match. (See, for example, the UnsortedTableMap.findIndex method in Code Fragment 10.4.) Therefore, when using a hash table to implement a map, we want equivalent keys to have the same hash code so that they are guaranteed to map to the same bucket. More formally, if a class defines equivalence through the equals method (see Section 3.5), then that class should also provide a consistent implementation of the hashCode method, such that if x.equals(y) then x.hashCode( ) == y.hashCode( ).

As an example, Java's String class defines the equals method so that two instances are equivalent if they have precisely the same sequence of characters. That class also overrides the hashCode method to provide consistent behavior. In fact, the implementation of hash codes for the String class is excellent. If we repeat the experiment from the previous page using Java's implementation of hash codes, there are only 12 collisions among more than 230,000 words. Java's primitive wrapper classes also define hashCode, using techniques described on page 412.

As an example of how to properly implement hashCode for a user-defined class, we will revisit the SinglyLinkedList class from Chapter 3. We defined the equals method for that class, in Section 3.5.2, so that two lists are equivalent if they represent equal-length sequences of elements that are pairwise equivalent. We can compute a robust hash code for a list by taking the exclusive-or of its elements' hash codes, while performing a cyclic shift. (See Code Fragment 10.6.)

```
1  public int hashCode( ) {
2    int h = 0;
3    for (Node walk=head; walk != null; walk = walk.getNext( )) {
4      h ^= walk.getElement( ).hashCode( );  // bitwise exclusive-or with element's code
5      h = (h << 5) | (h >>> 27);            // 5-bit cyclic shift of composite code
6    }
7    return h;
8  }
```

Code Fragment 10.6: A robust implementation of the hashCode method for the SinglyLinkedList class from Chapter 3.

Compression Functions

The hash code for a key k will typically not be suitable for immediate use with a bucket array, because the integer hash code may be negative or may exceed the capacity of the bucket array. Thus, once we have determined an integer hash code for a key object k, there is still the issue of mapping that integer into the range $[0, N-1]$. This computation, known as a ***compression function***, is the second action performed as part of an overall hash function. A good compression function is one that minimizes the number of collisions for a given set of distinct hash codes.

The Division Method

A simple compression function is the ***division method***, which maps an integer i to

$$i \bmod N,$$

where N, the size of the bucket array, is a fixed positive integer. Additionally, if we take N to be a prime number, then this compression function helps "spread out" the distribution of hashed values. Indeed, if N is not prime, then there is greater risk that patterns in the distribution of hash codes will be repeated in the distribution of hash values, thereby causing collisions. For example, if we insert keys with hash codes $\{200, 205, 210, 215, 220, \ldots, 600\}$ into a bucket array of size 100, then each hash code will collide with three others. But if we use a bucket array of size 101, then there will be no collisions. If a hash function is chosen well, it should ensure that the probability of two different keys getting hashed to the same bucket is $1/N$. Choosing N to be a prime number is not always enough, however, for if there is a repeated pattern of hash codes of the form $pN + q$ for several different p's, then there will still be collisions.

The MAD Method

A more sophisticated compression function, which helps eliminate repeated patterns in a set of integer keys, is the ***Multiply-Add-and-Divide*** (or "MAD") method. This method maps an integer i to

$$[(ai + b) \bmod p] \bmod N,$$

where N is the size of the bucket array, p is a prime number larger than N, and a and b are integers chosen at random from the interval $[0, p-1]$, with $a > 0$. This compression function is chosen in order to eliminate repeated patterns in the set of hash codes and get us closer to having a "good" hash function, that is, one such that the probability any two different keys collide is $1/N$. This good behavior would be the same as we would have if these keys were "thrown" into A uniformly at random.

10.2.2 Collision-Handling Schemes

The main idea of a hash table is to take a bucket array, A, and a hash function, h, and use them to implement a map by storing each entry (k,v) in the "bucket" $A[h(k)]$. This simple idea is challenged, however, when we have two distinct keys, k_1 and k_2, such that $h(k_1) = h(k_2)$. The existence of such ***collisions*** prevents us from simply inserting a new entry (k,v) directly into the bucket $A[h(k)]$. It also complicates our procedure for performing insertion, search, and deletion operations.

Separate Chaining

A simple and efficient way for dealing with collisions is to have each bucket $A[j]$ store its own secondary container, holding all entries (k,v) such that $h(k) = j$. A natural choice for the secondary container is a small map instance implemented using an unordered list, as described in Section 10.1.4. This ***collision resolution*** rule is known as ***separate chaining***, and is illustrated in Figure 10.6.

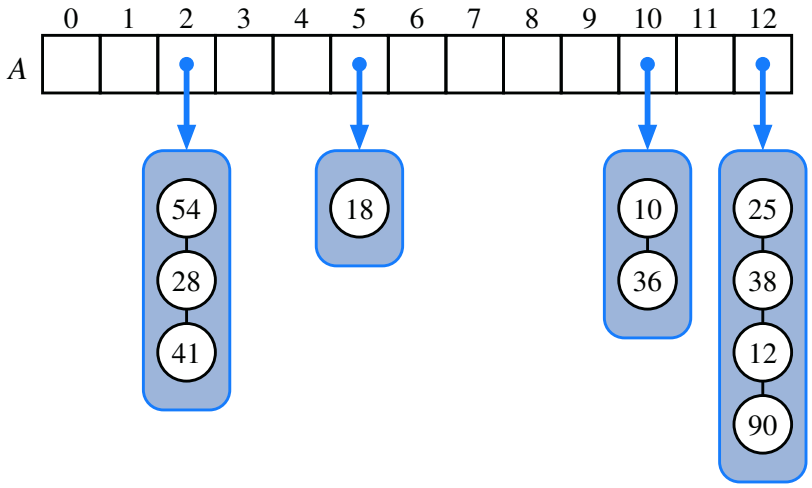

Figure 10.6: A hash table of size 13, storing 10 entries with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

In the worst case, operations on an individual bucket take time proportional to the size of the bucket. Assuming we use a good hash function to index the n entries of our map in a bucket array of capacity N, the expected size of a bucket is n/N. Therefore, if given a good hash function, the core map operations run in $O(\lceil n/N \rceil)$. The ratio $\lambda = n/N$, called the ***load factor*** of the hash table, should be bounded by a small constant, preferably below 1. As long as λ is $O(1)$, the core operations on the hash table run in $O(1)$ expected time.

Open Addressing

The separate chaining rule has many nice properties, such as affording simple implementations of map operations, but it nevertheless has one slight disadvantage: It requires the use of an auxiliary data structure to hold entries with colliding keys. If space is at a premium (for example, if we are writing a program for a small handheld device), then we can use the alternative approach of storing each entry directly in a table slot. This approach saves space because no auxiliary structures are employed, but it requires a bit more complexity to properly handle collisions. There are several variants of this approach, collectively referred to as ***open addressing*** schemes, which we discuss next. Open addressing requires that the load factor is always at most 1 and that entries are stored directly in the cells of the bucket array itself.

Linear Probing and Its Variants

A simple method for collision handling with open addressing is ***linear probing***. With this approach, if we try to insert an entry (k,v) into a bucket $A[j]$ that is already occupied, where $j = h(k)$, then we next try $A[(j+1) \bmod N]$. If $A[(j+1) \bmod N]$ is also occupied, then we try $A[(j+2) \bmod N]$, and so on, until we find an empty bucket that can accept the new entry. Once this bucket is located, we simply insert the entry there. Of course, this collision resolution strategy requires that we change the implementation when searching for an existing key—the first step of all get, put, or remove operations. In particular, to attempt to locate an entry with key equal to k, we must examine consecutive slots, starting from $A[h(k)]$, until we either find an entry with an equal key or we find an empty bucket. (See Figure 10.7.) The name "linear probing" comes from the fact that accessing a cell of the bucket array can be viewed as a "probe," and that consecutive probes occur in neighboring cells (when viewed circularly).

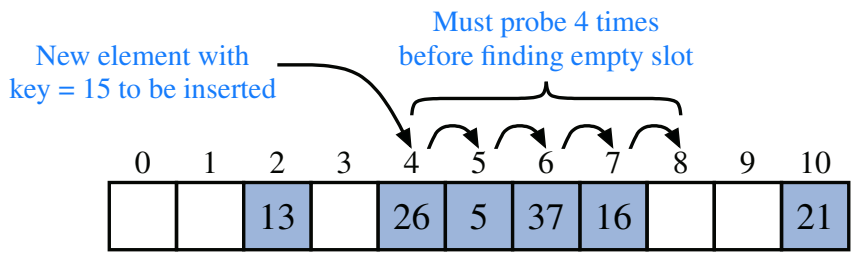

Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

To implement a deletion, we cannot simply remove a found entry from its slot in the array. For example, after the insertion of key 15 portrayed in Figure 10.7, if the entry with key 37 were trivially deleted, a subsequent search for 15 would fail because that search would start by probing at index 4, then index 5, and then index 6, at which an empty cell is found. A typical way to get around this difficulty is to replace a deleted entry with a special "defunct" sentinel object. With this special marker possibly occupying spaces in our hash table, we modify our search algorithm so that the search for a key k will skip over cells containing the defunct sentinel and continue probing until reaching the desired entry or an empty bucket (or returning back to where we started from). Additionally, our algorithm for put should remember a defunct location encountered during the search for k, since this is a valid place to put a new entry (k,v), if no existing entry is found beyond it.

Although use of an open addressing scheme can save space, linear probing suffers from an additional disadvantage. It tends to cluster the entries of a map into contiguous runs, which may even overlap (particularly if more than half of the cells in the hash table are occupied). Such contiguous runs of occupied hash cells cause searches to slow down considerably.

Another open addressing strategy, known as ***quadratic probing***, iteratively tries the buckets $A[(h(k)+f(i)) \bmod N]$, for $i = 0,1,2,\ldots$, where $f(i) = i^2$, until finding an empty bucket. As with linear probing, the quadratic probing strategy complicates the removal operation, but it does avoid the kinds of clustering patterns that occur with linear probing. Nevertheless, it creates its own kind of clustering, called ***secondary clustering***, where the set of filled array cells still has a nonuniform pattern, even if we assume that the original hash codes are distributed uniformly. When N is prime and the bucket array is less than half full, the quadratic probing strategy is guaranteed to find an empty slot. However, this guarantee is not valid once the table becomes at least half full, or if N is not chosen as a prime number; we explore the cause of this type of clustering in an exercise (C-10.42).

An open addressing strategy that does not cause clustering of the kind produced by linear probing or the kind produced by quadratic probing is the ***double hashing*** strategy. In this approach, we choose a secondary hash function, h', and if h maps some key k to a bucket $A[h(k)]$ that is already occupied, then we iteratively try the buckets $A[(h(k)+f(i)) \bmod N]$ next, for $i = 1,2,3,\ldots$, where $f(i) = i \cdot h'(k)$. In this scheme, the secondary hash function is not allowed to evaluate to zero; a common choice is $h'(k) = q - (k \bmod q)$, for some prime number $q < N$. Also, N should be a prime.

Another approach to avoid clustering with open addressing is to iteratively try buckets $A[(h(k)+f(i)) \bmod N]$ where $f(i)$ is based on a pseudorandom number generator, providing a repeatable, but somewhat arbitrary, sequence of subsequent probes that depends upon bits of the original hash code.

10.2.3 Load Factors, Rehashing, and Efficiency

In the hash table schemes described thus far, it is important that the load factor, $\lambda = n/N$, be kept below 1. With separate chaining, as λ gets very close to 1, the probability of a collision greatly increases, which adds overhead to our operations, since we must revert to linear-time list-based methods in buckets that have collisions. Experiments and average-case analyses suggest that we should maintain $\lambda < 0.9$ for hash tables with separate chaining. (By default, Java's implementation uses separate chaining with $\lambda < 0.75$.)

With open addressing, on the other hand, as the load factor λ grows beyond 0.5 and starts approaching 1, clusters of entries in the bucket array start to grow as well. These clusters cause the probing strategies to "bounce around" the bucket array for a considerable amount of time before they find an empty slot. In Exercise C-10.42, we explore the degradation of quadratic probing when $\lambda \geq 0.5$. Experiments suggest that we should maintain $\lambda < 0.5$ for an open addressing scheme with linear probing, and perhaps only a bit higher for other open addressing schemes.

If an insertion causes the load factor of a hash table to go above the specified threshold, then it is common to resize the table (to regain the specified load factor) and to reinsert all objects into this new table. Although we need not define a new hash code for each object, we do need to reapply a new compression function that takes into consideration the size of the new table. Rehashing will generally scatter the entries throughout the new bucket array. When rehashing to a new table, it is a good requirement for the new array's size to be a prime number approximately double the previous size (see Exercise C-10.32). In that way, the cost of rehashing all the entires in the table can be amortized against the time used to insert them in the first place (as with dynamic arrays; see Section 7.2.1).

Efficiency of Hash Tables

Although the details of the average-case analysis of hashing are beyond the scope of this book, its probabilistic basis is quite intuitive. If our hash function is good, then we expect the entries to be uniformly distributed in the N cells of the bucket array. Thus, to store n entries, the expected number of keys in a bucket would be $\lceil n/N \rceil$, which is $O(1)$ if n is $O(N)$.

The costs associated with a periodic rehashing (when resizing a table after occasional insertions or deletions) can be accounted for separately, leading to an additional $O(1)$ amortized cost for put and remove.

In the worst case, a poor hash function could map every entry to the same bucket. This would result in linear-time performance for the core map operations with separate chaining, or with any open addressing model in which the secondary sequence of probes depends only on the hash code. A summary of these costs is given in Table 10.2.

Method	Unsorted List	Hash Table	
		expected	worst case
get	$O(n)$	$O(1)$	$O(n)$
put	$O(n)$	$O(1)$	$O(n)$
remove	$O(n)$	$O(1)$	$O(n)$
size, isEmpty	$O(1)$	$O(1)$	$O(1)$
entrySet, keySet, values	$O(n)$	$O(n)$	$O(n)$

Table 10.2: Comparison of the running times of the methods of a map realized by means of an unsorted list (as in Section 10.1.4) or a hash table. We let n denote the number of entries in the map, and we assume that the bucket array supporting the hash table is maintained such that its capacity is proportional to the number of entries in the map.

An Anecdote About Hashing and Computer Security

In a 2003 academic paper, researchers discuss the possibility of exploiting a hash table's worst-case performance to cause a denial-of-service (DoS) attack of Internet technologies. Since many published algorithms compute hash codes with a deterministic function, an attacker could precompute a very large number of moderate-length strings that all hash to the identical 32-bit hash code. (Recall that by any of the hashing schemes we describe, other than double hashing, if two keys are mapped to the same hash code, they will be inseparable in the collision resolution.) This concern was brought to the attention of the Java development team, and that of many other programming languages, but deemed an insignificant risk at the time by most. (Kudos to the Perl team for implementing a fix in 2003.)

In late 2011, another team of researchers demonstrated an implementation of just such an attack. Web servers allow a series of key-value parameters to be embedded in a URL using a syntax such as `?key1=val1&key2=val2&key3=val3`. Those key-value pairs are strings and a typical Web server immediately stores them in a hash-map. Servers already place a limit on the length and number of such parameters, to avoid overload, but they presume that the total insertion time in the map will be linear in the number of entries, given the expected constant-time operations. However, if all keys were to collide, the insertions into the map will require quadratic time, causing the server to perform an inordinate amount of work.

In 2012, the OpenJDK team announced the following resolution: they distributed a security patch that includes an alternative hash function that introduces randomization into the computation of hash codes, making it less tractable to reverse engineer a set of colliding strings. However, to avoid breaking existing code, the new feature is disabled by default in Java SE 7 and, when enabled, is only used for hashing strings and only when a table size grows beyond a certain threshold. Enhanced hashing will be enabled in Java SE 8 for all types and uses.

10.2.4 Java Hash Table Implementation

In this section, we develop two implementations of a hash table, one using separate chaining and the other using open addressing with linear probing. While these approaches to collision resolution are quite different, there are many higher-level commonalities to the two hashing algorithms. For that reason, we extend the AbstractMap class (from Code Fragment 10.3) to define a new AbstractHashMap class (see Code Fragment 10.7), which provides much of the functionality common to our two hash table implementations.

We will begin by discussing what this abstract class does *not* do—it does not provide any concrete representation of a table of "buckets." With separate chaining, each bucket will be a secondary map. With open addressing, however, there is no tangible container for each bucket; the "buckets" are effectively interleaved due to the probing sequences. In our design, the AbstractHashMap class presumes the following to be abstract methods—to be implemented by each concrete subclass:

createTable( ): This method should create an initially empty table having size equal to a designated capacity instance variable.

bucketGet(h, k): This method should mimic the semantics of the public get method, but for a key k that is known to hash to bucket h.

bucketPut(h, k, v): This method should mimic the semantics of the public put method, but for a key k that is known to hash to bucket h.

bucketRemove(h, k): This method should mimic the semantics of the public remove method, but for a key k known to hash to bucket h.

entrySet( ): This standard map method iterates through *all* entries of the map. We do not delegate this on a per-bucket basis because "buckets" in open addressing are not inherently disjoint.

What the AbstractHashMap class does provide is mathematical support in the form of a hash compression function using a randomized Multiply-Add-and-Divide (MAD) formula, and support for automatically resizing the underlying hash table when the load factor reaches a certain threshold.

The hashValue method relies on an original key's hash code, as returned by its hashCode( ) method, followed by MAD compression based on a prime number and the scale and shift parameters that are randomly chosen in the constructor.

To manage the load factor, the AbstractHashMap class declares a protected member, n, which should equal the current number of entries in the map; however, it must rely on the subclasses to update this field from within methods bucketPut and bucketRemove. If the load factor of the table increases beyond 0.5, we request a bigger table (using the createTable method) and reinsert all entries into the new table. (For simplicity, this implementation uses tables of size $2^k + 1$, even though these are not generally prime.)

```
1  public abstract class AbstractHashMap<K,V> extends AbstractMap<K,V> {
2    protected int n = 0;                    // number of entries in the dictionary
3    protected int capacity;                 // length of the table
4    private int prime;                      // prime factor
5    private long scale, shift;              // the shift and scaling factors
6    public AbstractHashMap(int cap, int p) {
7      prime = p;
8      capacity = cap;
9      Random rand = new Random();
10     scale = rand.nextInt(prime-1) + 1;
11     shift = rand.nextInt(prime);
12     createTable();
13   }
14   public AbstractHashMap(int cap) { this(cap, 109345121); }   // default prime
15   public AbstractHashMap() { this(17); }                      // default capacity
16   // public methods
17   public int size() { return n; }
18   public V get(K key) { return bucketGet(hashValue(key), key); }
19   public V remove(K key) { return bucketRemove(hashValue(key), key); }
20   public V put(K key, V value) {
21     V answer = bucketPut(hashValue(key), key, value);
22     if (n > capacity / 2)                 // keep load factor <= 0.5
23       resize(2 * capacity - 1);           // (or find a nearby prime)
24     return answer;
25   }
26   // private utilities
27   private int hashValue(K key) {
28     return (int) ((Math.abs(key.hashCode()*scale + shift) % prime) % capacity);
29   }
30   private void resize(int newCap) {
31     ArrayList<Entry<K,V>> buffer = new ArrayList<>(n);
32     for (Entry<K,V> e : entrySet())
33       buffer.add(e);
34     capacity = newCap;
35     createTable();                        // based on updated capacity
36     n = 0;                                // will be recomputed while reinserting entries
37     for (Entry<K,V> e : buffer)
38       put(e.getKey(), e.getValue());
39   }
40   // protected abstract methods to be implemented by subclasses
41   protected abstract void createTable();
42   protected abstract V bucketGet(int h, K k);
43   protected abstract V bucketPut(int h, K k, V v);
44   protected abstract V bucketRemove(int h, K k);
45 }
```

Code Fragment 10.7: A base class for our hash table implementations, extending the AbstractMap class from Code Fragment 10.3.

Separate Chaining

To represent each bucket for separate chaining, we use an instance of the simpler UnsortedTableMap class from Section 10.1.4. This technique, in which we use a simple solution to a problem to create a new, more advanced solution, is known as ***bootstrapping***. The advantage of using a map for each bucket is that it becomes easy to delegate responsibilities for top-level map operations to the appropriate bucket.

The entire hash table is then represented as a fixed-capacity array A of the secondary maps. Each cell, $A[h]$, is initially a null reference; we only create a secondary map when an entry is first hashed to a particular bucket.

As a general rule, we implement bucketGet(h, k) by calling $A[h]$.get(k), we implement bucketPut(h, k, v) by calling $A[h]$.put(k, v), and bucketRemove(h, k) by calling $A[h]$.remove(k). However, care is needed for two reasons.

First, because we choose to leave table cells as null until a secondary map is needed, each of these fundamental operations must begin by checking to see if $A[h]$ is null. In the case of bucketGet and bucketRemove, if the bucket does not yet exist, we can simply return null as there can not be any entry matching key k. In the case of bucketPut, a new entry must be inserted, so we instantiate a new UnsortedTableMap for $A[h]$ before continuing.

The second issue is that, in our AbstractHashMap framework, the subclass has the responsibility to properly maintain the instance variable n when an entry is newly inserted or deleted. Remember that when put(k, v) is called on a map, the size of the map only increases if key k is new to the map (otherwise, the value of an existing entry is reassigned). Similarly, a call to remove(k) only decreases the size of the map when an entry with key equal to k is found. In our implementation, we determine the change in the overall size of the map, by determining if there is any change in the size of the relevant secondary map before and after an operation.

Code Fragment 10.8 provides a complete definition for our ChainHashMap class, which implements a hash table with separate chaining. If we assume that the hash function performs well, a map with n entries and a table of capacity N will have an expected bucket size of n/N (recall, this is its ***load factor***). So even though the individual buckets, implemented as UnsortedTableMap instances, are not particularly efficient, each bucket has expected $O(1)$ size, provided that n is $O(N)$, as in our implementation. Therefore, the expected running time of operations get, put, and remove for this map is $O(1)$. The entrySet method (and thus the related keySet and values) runs in $O(n+N)$ time, as it loops through the length of the table (with length N) and through all buckets (which have cumulative lengths n).

```
1  public class ChainHashMap<K,V> extends AbstractHashMap<K,V> {
2    // a fixed capacity array of UnsortedTableMap that serve as buckets
3    private UnsortedTableMap<K,V>[ ] table;   // initialized within createTable
4    public ChainHashMap( ) { super( ); }
5    public ChainHashMap(int cap) { super(cap); }
6    public ChainHashMap(int cap, int p) { super(cap, p); }
7    /** Creates an empty table having length equal to current capacity. */
8    protected void createTable( ) {
9      table = (UnsortedTableMap<K,V>[ ]) new UnsortedTableMap[capacity];
10   }
11   /** Returns value associated with key k in bucket with hash value h, or else null. */
12   protected V bucketGet(int h, K k) {
13     UnsortedTableMap<K,V> bucket = table[h];
14     if (bucket == null) return null;
15     return bucket.get(k);
16   }
17   /** Associates key k with value v in bucket with hash value h; returns old value. */
18   protected V bucketPut(int h, K k, V v) {
19     UnsortedTableMap<K,V> bucket = table[h];
20     if (bucket == null)
21       bucket = table[h] = new UnsortedTableMap<>( );
22     int oldSize = bucket.size( );
23     V answer = bucket.put(k,v);
24     n += (bucket.size( ) − oldSize);      // size may have increased
25     return answer;
26   }
27   /** Removes entry having key k from bucket with hash value h (if any). */
28   protected V bucketRemove(int h, K k) {
29     UnsortedTableMap<K,V> bucket = table[h];
30     if (bucket == null) return null;
31     int oldSize = bucket.size( );
32     V answer = bucket.remove(k);
33     n −= (oldSize − bucket.size( ));      // size may have decreased
34     return answer;
35   }
36   /** Returns an iterable collection of all key-value entries of the map. */
37   public Iterable<Entry<K,V>> entrySet( ) {
38     ArrayList<Entry<K,V>> buffer = new ArrayList<>( );
39     for (int h=0; h < capacity; h++)
40       if (table[h] != null)
41         for (Entry<K,V> entry : table[h].entrySet( ))
42           buffer.add(entry);
43     return buffer;
44   }
45 }
```

Code Fragment 10.8: A concrete hash map implementation using separate chaining.

Linear Probing

Our implementation of a ProbeHashMap class, using open addressing with linear probing, is given in Code Fragments 10.9 and 10.10. In order to support deletions, we use a technique described in Section 10.2.2 in which we place a special marker in a table location at which an entry has been deleted, so that we can distinguish between it and a location that has always been empty. To this end, we create a fixed entry instance, DEFUNCT, as a sentinel (disregarding any key or value stored within), and use references to that instance to mark vacated cells.

The most challenging aspect of open addressing is to properly trace the series of probes when collisions occur during a search for an existing entry, or placement of a new entry. To this end, the three primary map operations each rely on a utility, findSlot, that searches for an entry with key k in "bucket" h (that is, where h is the index returned by the hash function for key k). When attempting to retrieve the value associated with a given key, we must continue probing until we find the key, or until we reach a table slot with a null reference. We cannot stop the search upon reaching an DEFUNCT sentinel, because it represents a location that may have been filled at the time the desired entry was once inserted.

When a key-value pair is being placed in the map, we must first attempt to find an existing entry with the given key, so that we might overwrite its value. Therefore, we must search beyond any occurrences of the DEFUNCT sentinel when inserting. However, if no match is found, we prefer to repurpose the first slot marked with DEFUNCT, if any, when placing the new element in the table. The findSlot method enacts this logic, continuing an unsuccessful search until finding a truly empty slot, and returning the index of the first available slot for an insertion.

When deleting an existing entry within bucketRemove, we intentionally set the table entry to the DEFUNCT sentinel in accordance with our strategy.

```
1  public class ProbeHashMap<K,V> extends AbstractHashMap<K,V> {
2    private MapEntry<K,V>[ ] table;        // a fixed array of entries (all initially null)
3    private MapEntry<K,V> DEFUNCT = new MapEntry<>(null, null);   //sentinel
4    public ProbeHashMap( ) { super( ); }
5    public ProbeHashMap(int cap) { super(cap); }
6    public ProbeHashMap(int cap, int p) { super(cap, p); }
7    /** Creates an empty table having length equal to current capacity. */
8    protected void createTable( ) {
9      table = (MapEntry<K,V>[ ]) new MapEntry[capacity];   // safe cast
10   }
11   /** Returns true if location is either empty or the "defunct" sentinel. */
12   private boolean isAvailable(int j) {
13     return (table[j] == null || table[j] == DEFUNCT);
14   }
```

Code Fragment 10.9: Concrete ProbeHashMap class that uses linear probing for collision resolution. (Continues in Code Fragment 10.10.)

```
15    /** Returns index with key k, or -(a+1) such that k could be added at index a. */
16    private int findSlot(int h, K k) {
17      int avail = -1;                          // no slot available (thus far)
18      int j = h;                               // index while scanning table
19      do {
20        if (isAvailable(j)) {                  // may be either empty or defunct
21          if (avail == -1) avail = j;          // this is the first available slot!
22          if (table[j] == null) break;         // if empty, search fails immediately
23        } else if (table[j].getKey( ).equals(k))
24          return j;                            // successful match
25        j = (j+1) % capacity;                  // keep looking (cyclically)
26      } while (j != h);                        // stop if we return to the start
27      return -(avail + 1);                     // search has failed
28    }
29    /** Returns value associated with key k in bucket with hash value h, or else null. */
30    protected V bucketGet(int h, K k) {
31      int j = findSlot(h, k);
32      if (j < 0) return null;                  // no match found
33      return table[j].getValue( );
34    }
35    /** Associates key k with value v in bucket with hash value h; returns old value. */
36    protected V bucketPut(int h, K k, V v) {
37      int j = findSlot(h, k);
38      if (j >= 0)                              // this key has an existing entry
39        return table[j].setValue(v);
40      table[-(j+1)] = new MapEntry<>(k, v);    // convert to proper index
41      n++;
42      return null;
43    }
44    /** Removes entry having key k from bucket with hash value h (if any). */
45    protected V bucketRemove(int h, K k) {
46      int j = findSlot(h, k);
47      if (j < 0) return null;                  // nothing to remove
48      V answer = table[j].getValue( );
49      table[j] = DEFUNCT;                      // mark this slot as deactivated
50      n--;
51      return answer;
52    }
53    /** Returns an iterable collection of all key-value entries of the map. */
54    public Iterable<Entry<K,V>> entrySet( ) {
55      ArrayList<Entry<K,V>> buffer = new ArrayList<>( );
56      for (int h=0; h < capacity; h++)
57        if (!isAvailable(h)) buffer.add(table[h]);
58      return buffer;
59    }
60  }
```

Code Fragment 10.10: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued from Code Fragment 10.9).